# AuraBake

*Smart Campus Pre-Order Food System*

## Project Proposal & Technical Documentation

Submitted to College Management

| | |
|---|---|
| **Project Name** | AuraBake — College Pre-Order Food System |
| **Technology Stack** | FastAPI · PostgreSQL · JWT · React/Mobile |
| **Document Type** | Full Project Proposal & Technical Architecture |
| **Version** | v1.0 — Initial Release |
| **Date** | April 2026 |

# 1. Executive Summary

AuraBake is a smart, time-slot-based food pre-ordering system designed specifically for college campuses. This proposal presents a comprehensive solution to one of the most persistent daily problems in academic institutions — the chaotic lunch break queue at the campus canteen.

Every day, hundreds of students rush to the canteen during limited break windows, causing overcrowding, delayed food preparation, frequent stock-outs, and significant student stress. AuraBake directly addresses this through a structured digital ordering and scheduling system that distributes demand intelligently across defined time slots.

## 1.1 The Problem

- Students lose 20–30 minutes per day standing in unmanaged queues
- Kitchen staff face unpredictable demand surges with no advance planning data
- Popular items sell out while others go to waste due to lack of forecasting
- No real-time visibility into orders — staff must manually track everything
- Students frequently miss class or skip meals entirely during peak hours

## 1.2 The AuraBake Solution

AuraBake replaces chaotic walk-in ordering with a structured digital pre-order model. Students browse a live menu, select a pickup time slot, place and pay for their order in advance — then simply arrive at the dedicated counter at their scheduled time to collect their food. No queues. No waiting. No uncertainty.

## 1.3 Key Benefits at a Glance

| Stakeholder | Primary Benefit |
|---|---|
| Students | Zero wait time — collect pre-ordered food at chosen slot |
| Bakers / Kitchen Staff | Advance order visibility, balanced workload, no rush panic |
| Administrators | Full order analytics, revenue tracking, user management |
| College Institution | Improved campus experience, reduced food waste, operational data |

# 2. Project Overview

## 2.1 Application Identity

**Application Name:** AuraBake
**Tagline:** Baked Fresh. Ordered Smart.
**Platform:** Mobile Application (Android/iOS) + Web Portal
**Backend Framework:** Python FastAPI
**Database:** PostgreSQL
**Authentication:** JWT (JSON Web Tokens) with bcrypt password hashing
**Architecture Pattern:** 3-Tier Layered Architecture

## 2.2 System Roles

AuraBake supports three distinct user roles, each with a tailored interface and permission set:

| Role | Description | Key Capabilities |
|---|---|---|
| Customer (Student) | Primary end-users of the platform | Register, browse menu, select time slots, place orders, make payments, track status |
| Baker | Canteen kitchen staff | View incoming orders by slot, update preparation status, manage food availability |
| Administrator | System managers | Manage users & bakers, configure time slots, monitor all orders, generate reports |

## 2.3 Core Value Proposition

The defining innovation of AuraBake is its Time Slot Allocation Algorithm — a database-level concurrency-safe mechanism that ensures no single time window accepts more orders than the kitchen can prepare. This is not merely a booking system; it is an intelligent capacity management engine.

# 3. Problem Statement & Justification

## 3.1 The Queue Crisis in College Canteens

College canteens serve hundreds of students within narrow 30-45 minute lunch windows. The traditional walk-in model creates a phenomenon known as demand bunching — all users arrive simultaneously, overwhelming both the serving counter and the kitchen staff.

Academic research and on-ground observation consistently reveal the following impact metrics in similar institutions:

| Problem Area | Observable Impact | Consequence |
|---|---|---|
| Queue Wait Time | 15–30 minutes average | Students eat late or skip meals |
| Kitchen Demand Surge | Unpredictable peak load | Food errors, delays, staff burnout |
| Inventory Management | No advance demand data | Overproduction waste / stock-outs |
| Order Accuracy | Verbal/manual order taking | High error rate during rush |
| Student Wellbeing | Stress, missed classes | Reduced academic performance |

## 3.2 Why a Scheduling Solution Works

The core insight behind AuraBake is that the canteen queue problem is fundamentally a scheduling problem, not a food production problem. By distributing demand across 10-minute time slots, the system transforms a chaotic surge into a predictable, evenly-spaced stream of orders.

This is the same principle used in hospital appointment systems, railway ticketing, and airline check-in management — all of which successfully eliminate queues through advance slot allocation. AuraBake applies this proven computing principle to the campus canteen context.

# 4. System Architecture

## 4.1 High-Level Architecture

AuraBake follows a 3-Tier Layered Architecture ensuring clear separation of concerns between the frontend presentation layer, backend business logic, and data persistence layer.

*Figure 4.1 — Component Diagram: System Architecture*

## 4.2 Backend Layer Structure

The FastAPI backend is organized into distinct functional layers:

| Layer | Directory | Responsibility |
|---|---|---|
| API Layer | routers/ | Defines HTTP endpoints and routes requests |
| Business Logic Layer | services/ | Implements core application logic and workflows |
| Database Layer | models/ | SQLAlchemy ORM models for database interaction |
| Validation Layer | schemas/ | Pydantic schemas for request/response validation |
| Security Layer | core/ | JWT generation, bcrypt hashing, RBAC dependencies |
| Helper Layer | utils/ | Shared utilities, email service, formatters |
| DB Connection Layer | db/ | Session management, connection pooling |

## 4.3 Full System Activity Diagram

The following diagram illustrates the complete system workflow across all three user roles — Customer, Baker, and Administrator:

*Figure 4.2 — Full System Activity Diagram (All Roles)*

## 4.4 Technology Stack Summary

| Component | Technology | Justification |
|---|---|---|
| Backend API | Python + FastAPI | High performance, async support, auto-generated API docs |
| Database | PostgreSQL | ACID compliance, row-level locking for slot |

| Component | Technology | Justification |
|---|---|---|
| | | concurrency |
| Authentication | JWT + bcrypt | Stateless auth, industry-standard security |
| Frontend | React / React Native | Cross-platform, component-based UI |
| API Testing | Postman + PyTest | Comprehensive unit and integration testing |
| Load Testing | Apache JMeter | Validate peak-hour performance |

# 5. User Role Specifications

## 5.1 Customer (Student) Workflow

Students are the primary beneficiaries of AuraBake. The customer journey is designed to be intuitive, fast, and completely queue-free.

*Figure 5.1 — Customer Activity Diagram*

### Customer Capabilities:

- Register and login with secure JWT authentication
- Browse live digital menu with real-time availability indicators
- Select a preferred pickup time slot from available windows
- Place orders with item quantity selection
- Make online payment or choose cash-on-pickup
- Receive real-time notifications for order status updates
- Track order from Confirmed → Preparing → Ready → Completed
- Cancel orders before preparation begins (with auto-refund for online payments)

## 5.2 Baker Workflow

Bakers interact with AuraBake through a streamlined dashboard that organizes all incoming orders by time slot, enabling systematic, stress-free preparation.

*Figure 5.2 — Baker Activity Diagram*

### Baker Capabilities:

- Secure login to dedicated baker dashboard
- View all orders grouped and sorted by pickup time slot
- Verify payment status before beginning food preparation
- Update order status: Preparing → Ready → Completed
- Trigger customer notification when order is ready for pickup
- Verify physical pickup and mark orders as completed
- Manage menu item availability and quantities

## 5.3 Administrator Workflow

Administrators have complete oversight and control of the entire AuraBake ecosystem, from user management to analytics reporting.

*Figure 5.3 — Administrator Activity Diagram*

### Administrator Capabilities:

- **Activate, deactivate, or remove student accounts:** Manage Users
- **Add or suspend baker accounts, assign menu items:** Manage Bakers
- **Create, edit, and validate time slot schedules with overlap detection:** Configure Time Slots
- **Real-time view of all orders across all slots with force-cancel/refund capability:** Monitor Orders
- **Date-range analytics with exportable revenue and usage reports:** Generate Reports

# 6. Order Lifecycle & Placement Flow

## 6.1 Order State Machine

Every order in AuraBake progresses through a well-defined state machine. State transitions are strictly controlled and logged for audit purposes.

*Figure 6.1 — Order Lifecycle State Diagram*

| State | Description | Triggered By |
|---|---|---|
| Created | Order record initialized in database | Customer submits order |
| Payment_Pending | Awaiting online payment confirmation | System after order creation |
| Confirmed | Payment verified, order accepted | Payment gateway callback |
| Preparing | Baker has begun food preparation | Baker updates status |
| Ready | Food prepared, awaiting customer pickup | Baker marks ready |
| Completed | Customer collected the order | Baker verifies pickup |
| Cancelled | Order cancelled (pre-preparation) | Customer request or payment failure |
| Expired | Order not collected within slot window | System auto-expiry |

## 6.2 Order Placement Sequence

The order placement flow involves coordinated interactions between the customer app, backend API, database, and payment gateway:

*Figure 6.2 — Order Placement Activity Diagram*

*Figure 6.3 — System-Level Order Placement Sequence Diagram*

### Order Placement Steps:

1. Customer selects items and desired pickup time slot
2. App sends order request to backend with slot ID and item details

3. Backend checks stock availability and slot capacity simultaneously
4. Payment gateway is invoked for online payments
5. On payment success: order created, inventory reduced, slot count incremented
6. Baker receives real-time notification of new order
7. Customer receives order confirmation with pickup details

# 7. Time Slot Allocation Algorithm

The Time Slot Allocation Algorithm is the defining technical innovation of AuraBake. It is what transforms a simple ordering app into a genuine queue-elimination system.

## 7.1 Problem Definition

In a traditional system, two students could simultaneously view the same time slot as "available" and both submit orders — resulting in overbooking. This is a classic race condition in concurrent database access, and it directly undermines the reliability of the slot system.

## 7.2 TimeSlot Data Model

| Field | Data Type | Description |
|---|---|---|
| slot_id | UUID (PK) | Unique identifier for each time slot |
| start_time | DATETIME | Slot window opening time (e.g., 12:30 PM) |
| end_time | DATETIME | Slot window closing time (e.g., 12:40 PM) |
| max_capacity | INTEGER | Maximum orders this slot can accept |
| current_orders | INTEGER | Count of currently confirmed orders |
| status | ENUM | OPEN / FULL / CLOSED — real-time availability state |

## 7.3 Concurrency-Safe Algorithm

AuraBake uses PostgreSQL row-level locking via SELECT FOR UPDATE within a database transaction to prevent race conditions:

```
BEGIN TRANSACTION  →  Lock slot row for this transaction
SELECT current_orders FROM TimeSlot WHERE slot_id = X FOR UPDATE
IF current_orders < max_capacity:
   INSERT Order record
   UPDATE current_orders = current_orders + 1
   COMMIT
ELSE:
   ROLLBACK  →  Return 'Slot Full' error to user
```

## 7.4 Edge Cases Handled

- User selects slot but delays payment — temporary reservation released on timeout
- Order cancellation — slot count decremented atomically to free capacity

- Admin reduces max_capacity for a slot — current bookings preserved, new bookings blocked if full
- Overlapping slot definitions — admin UI validates for overlap before saving
- Double-click order submission — idempotency key prevents duplicate order creation

## 7.5 Advanced Dynamic Capacity Formula

In a future enhancement phase, static max_capacity will be replaced by a dynamic formula:

***capacity = (slot_duration_minutes / avg_prep_time_minutes) × number_of_active_staff***

This will allow the system to automatically adjust slot capacity based on real-time staffing levels and item complexity.

# 8. Database Design

## 8.1 Entity-Relationship Diagram

*Figure 8.1 — ER Diagram: Database Model*

## 8.2 Core Entities

### User Table

| Field | Type | Constraints | Description |
|---|---|---|---|
| user_id | UUID | Primary Key | Unique user identifier |
| name | VARCHAR(100) | NOT NULL | Full name of the user |
| email | VARCHAR(255) | UNIQUE, NOT NULL | Login email address |
| password_hash | VARCHAR(255) | NOT NULL | bcrypt hashed password |
| role | ENUM | NOT NULL | customer / baker / admin |
| is_active | BOOLEAN | DEFAULT TRUE | Account activation status |
| created_at | TIMESTAMP | DEFAULT NOW() | Account creation timestamp |

### Order Table

| Field | Type | Constraints | Description |
|---|---|---|---|
| order_id | UUID | Primary Key | Unique order identifier |
| user_id | UUID | Foreign Key → User | Customer who placed the order |
| item_id | UUID | Foreign Key → Menu | Ordered menu item |
| slot_id | UUID | Foreign Key → TimeSlot | Selected pickup time slot |
| quantity | INTEGER | NOT NULL, > 0 | Number of items ordered |
| status | ENUM | NOT NULL | Order lifecycle state |
| total_price | DECIMAL(10,2) | NOT NULL | Total cost of order |
| payment_status | ENUM | NOT NULL | pending / paid / refunded |
| created_at | TIMESTAMP | DEFAULT NOW() | Order creation time |

### Menu Table

| Field | Type | Constraints | Description |
|---|---|---|---|
| item_id | UUID | Primary Key | Unique item identifier |
| name | VARCHAR(150) | NOT NULL | Name of the food item |
| price | DECIMAL(8,2) | NOT NULL | Price per unit |
| quantity | INTEGER | NOT NULL | Current available stock |
| baker_id | UUID | Foreign Key → User | Baker responsible for this item |

# 9. Security Design

## 9.1 Security Objectives

AuraBake is designed with security as a first-class concern. The system addresses six core security principles:

| Principle | Implementation |
|---|---|
| Confidentiality | All user data encrypted; passwords never stored in plain text |
| Integrity | Database transactions ensure order and payment data cannot be partially modified |
| Availability | Load-tested for peak concurrent users during lunch hours |
| Authentication | JWT tokens with configurable expiry; bcrypt password hashing |
| Authorization | Role-Based Access Control (RBAC) middleware on every protected endpoint |
| Non-repudiation | Full audit trail of all order state transitions with timestamps |

## 9.2 Threat Model & Mitigations

| Threat | Risk Level | Mitigation Strategy |
|---|---|---|
| SQL Injection | HIGH | SQLAlchemy ORM with parameterized queries; no raw SQL |
| Password Theft | HIGH | bcrypt hashing with cost factor 12–14; never logged |
| JWT Token Hijacking | HIGH | Short expiry (15 min access + refresh tokens); HTTPS only |
| Role Escalation | CRITICAL | Server-side RBAC on every endpoint; client role is never trusted |
| MITM Attack | HIGH | Mandatory HTTPS/TLS for all communications |
| Brute Force | MEDIUM | Rate limiting on login endpoint; account lockout after 5 attempts |
| CSRF | MEDIUM | JWT-based auth (stateless) inherently resistant to CSRF |
| XSS | MEDIUM | Input sanitization; Content Security Policy headers |

## 9.3 Authentication Flow

### Password Hashing (bcrypt)

- Registration: plain_password → bcrypt.hash(cost=12) → store hashed_password
- Login: input_password → bcrypt.verify(stored_hash) → grant/deny access

- Passwords are never logged, never transmitted after registration, never stored plain

### JWT Token Architecture

- Access Token: Short-lived (15 minutes), contains user_id, role, expiry
- Refresh Token: Long-lived (7 days), used to obtain new access tokens without re-login
- All protected endpoints validate token signature and role before processing

# 10. API Design

## 10.1 Authentication Endpoints

| Method | Endpoint | Access | Description |
|---|---|---|---|
| POST | /auth/register | Public | Register new user with role assignment |
| POST | /auth/login | Public | Authenticate and receive JWT tokens |
| POST | /auth/refresh | Authenticated | Obtain new access token using refresh token |
| POST | /auth/logout | Authenticated | Invalidate current session tokens |

## 10.2 Order Management Endpoints

| Method | Endpoint | Access | Description |
|---|---|---|---|
| POST | /orders/create | Customer | Create new order with slot and item selection |
| GET | /orders/my-orders | Customer | Retrieve all orders for authenticated customer |
| GET | /orders/by-slot/{slot_id} | Baker | View all orders for a specific time slot |
| PUT | /orders/status/{order_id} | Baker/Admin | Update order preparation status |
| DELETE | /orders/{order_id} | Customer/ Admin | Cancel order (pre-preparation only) |

## 10.3 Slot Management Endpoints

| Method | Endpoint | Access | Description |
|---|---|---|---|
| GET | /slots/available | Customer | List all currently open time slots |
| POST | /slots/create | Admin | Create new time slot with capacity setting |
| PUT | /slots/{slot_id} | Admin | Edit slot timing or capacity |
| DELETE | /slots/{slot_id} | Admin | Remove a time slot (if no active orders) |

## 10.4 Payment Endpoints

| Method | Endpoint | Access | Description |
|---|---|---|---|
| POST | /payment/initiate | Customer | Initiate online payment session for an order |

| Method | Endpoint | Access | Description |
|---|---|---|---|
| POST | /payment/verify | System/ Webhook | Verify signed payment callback from gateway |
| POST | /payment/upi/initiate | Customer | Generate UPI deep link intent (Phase 2) |
| GET | /payment/qr/{order_id} | Customer | Generate UPI QR code image for payment (Phase 2) |
| POST | /payment/refund/{order_id} | Admin | Trigger refund for cancelled or failed order |
| GET | /payment/status/{order_id} | Customer/ Admin | Get current payment status of an order |

# 11. Payment Gateway Integration

AuraBake implements a multi-mode payment architecture designed for immediate online card/wallet payments, with a structured roadmap to native UPI and QR/barcode-based payment flows. The payment system is built as an independent module so new payment methods can be plugged in without touching order logic.

## 11.1 Payment Architecture Overview

| Payment Mode | Status | Method | User Experience |
|---|---|---|---|
| Cash on Pickup | Live (Phase 1) | Manual | Order confirmed; customer pays at counter |
| Online Card / Wallet | Live (Phase 1) | Payment Gateway API | Redirect to gateway; callback confirms order |
| UPI (VPA / Intent) | Phase 2 Roadmap | UPI Intent API | App-to-app UPI deep link; instant confirmation |
| UPI QR Code | Phase 2 Roadmap | Dynamic QR Generation | Customer scans QR in any UPI app to pay |
| Barcode (Order Pickup) | Phase 3 Roadmap | Barcode / QR Scan | Baker scans code at counter to verify and complete |

## 11.2 Phase 1 — Online Payment Gateway (Current)

AuraBake integrates with a standard payment gateway (Razorpay / Stripe / PayU) using their REST API and webhook callback pattern. This is the most widely-adopted integration model and is fully functional in Phase 1.

### Payment Flow (Online Card / Wallet):

8. Customer confirms order and selects Pay Online
9. Backend creates a payment order via gateway API (POST /payment/initiate)
10. Customer is redirected to the gateway secure hosted checkout page
11. Customer completes payment (card / netbanking / wallet)
12. Gateway sends a signed webhook POST to /payment/verify on the backend
13. Backend verifies webhook signature (HMAC-SHA256), updates order to Confirmed
14. Slot count incremented, baker notified, customer receives confirmation

### Payment Database Schema:

| Field | Type | Description |
|---|---|---|
| payment_id | UUID (PK) | Unique payment record identifier |

| Field | Type | Description |
|---|---|---|
| order_id | UUID (FK) | Associated order reference |
| gateway_order_id | VARCHAR | Order ID assigned by payment gateway |
| gateway_payment_id | VARCHAR | Transaction ID from gateway on success |
| method | ENUM | cash / card / netbanking / wallet / upi / qr |
| amount | DECIMAL(10,2) | Total amount charged |
| currency | VARCHAR(3) | Currency code (default: INR) |
| status | ENUM | pending / success / failed / refunded |
| webhook_verified | BOOLEAN | Whether webhook signature was validated |
| created_at | TIMESTAMP | Payment initiation timestamp |
| updated_at | TIMESTAMP | Last status update timestamp |

**Refund Flow:**

- Customer or Admin requests cancellation before order reaches Preparing state
- Backend calls gateway refund API with original gateway_payment_id
- Refund webhook received; payment status updated to refunded
- Slot count decremented; customer receives refund notification
- Cash orders: Admin manually processes refund; system logs the action

## 11.3 Phase 2 — UPI Integration

UPI (Unified Payments Interface) is India most widely-used digital payment method with over 300 million active users. Phase 2 integrates native UPI through two flows: UPI Intent (deep link) and UPI QR Code.

### 11.3.1 UPI Intent (Deep Link)

The UPI Intent method launches a UPI app directly on the customer device — the fastest user experience for mobile orders.

| Step | Action | Technical Detail |
|---|---|---|
| 1 | Customer taps Pay via UPI | App calls POST /payment/upi/initiate |
| 2 | Backend generates UPI intent | Creates upi://pay?pa=...&am=...&tr=... deep link |
| 3 | App opens UPI intent chooser | Android/iOS shows installed UPI apps |
| 4 | Customer selects app and | PhonePe / GPay / Paytm / BHIM etc. |

| Step | Action | Technical Detail |
|---|---|---|
| | authenticates | |
| 5 | Payment completed in UPI app | Transaction ID returned to AuraBake |
| 6 | Backend verifies via gateway | NPCI transaction verified server-side |
| 7 | Order confirmed | Slot locked, baker notified, confirmation sent |

### 11.3.2 UPI QR Code

Dynamic UPI QR codes are generated per-order, allowing customers to pay using any UPI-compatible app by scanning. This is especially useful for users transitioning from cash to digital payments.

- Backend generates a unique QR code per order using the UPI deep link URI
- QR is encoded and served via GET /payment/qr/{order_id} as an inline image
- Customer scans QR with PhonePe, Google Pay, Paytm, BHIM, or any UPI app
- Payment success triggers the standard webhook verification flow
- QR codes expire after 10 minutes for security; a new QR can be requested

| UPI Component | Technology |
|---|---|
| UPI Intent Generation | Razorpay UPI / PayU UPI / NPCI UPI SDK |
| QR Code Generation | Python qrcode library (server-side rendering) |
| QR Image Format | PNG served inline or as base64 data URI |
| VPA (Virtual Payment Address) | Canteen registered UPI ID (e.g., aurabake@upi) |
| Transaction Verification | Gateway webhook + NPCI status API cross-check |

## 11.4 Phase 3 — Barcode-Based Order Pickup Verification

In Phase 3, AuraBake extends the QR/barcode concept from payment into the physical pickup experience. Every confirmed order carries a unique scannable code enabling contactless, error-free pickup verification at the counter.

### Pickup Flow with Order QR Code:

15. Once payment is confirmed, system generates a signed Order QR Code
16. QR is displayed in the customer app on the My Orders screen
17. Baker uses the AuraBake Baker app to scan the QR at the pickup counter
18. Backend verifies: correct order, valid slot time window, payment confirmed

19. Order status instantly updated to Completed; audit log entry created

| Benefit | Description |
|---|---|
| Zero Manual Errors | Eliminates verbal name-calling or token confusion at counter |
| Fraud Prevention | QR contains signed JWT token — cannot be duplicated or reused |
| Speed | Scan-and-go pickup takes under 3 seconds at the counter |
| Audit Trail | Every pickup timestamped and logged against baker ID |
| Expiry Control | QR becomes invalid after slot window closes — no late pickups |

**QR Security Design:**

- QR payload is a signed JWT: { order_id, user_id, slot_id, exp } — signed with server secret
- One-time use: scanning marks QR as used in database; rescan returns Already Collected
- Slot-window enforcement: QR only valid within +/- 5 minutes of pickup slot time
- Baker app requires authentication before scan functionality is unlocked

## 11.5 Payment Mode Comparison Summary

| Feature | Cash | Card/Wallet | UPI Intent | UPI QR | Barcode Pickup |
|---|---|---|---|---|---|
| Phase | 1 | 1 | 2 | 2 | 3 |
| User Action | Pay at counter | Enter card details | Tap in UPI app | Scan & pay | Show QR to baker |
| Internet Required | No | Yes | Yes | Yes | Yes |
| Refund Support | Manual | Automatic | Automatic | Automatic | N/A |
| Fraud Risk | Medium | Low | Low | Low | Very Low |
| Speed | Slow (queue) | Medium | Fast | Fast | Instant |

# 12. Testing Strategy

AuraBake follows a multi-level testing approach to ensure correctness, performance, and security across all system components.

## 11.1 Testing Levels

| Level | Tool | Scope |
|---|---|---|
| Unit Testing | PyTest | Individual functions: auth, slot validation, order logic |
| Integration Testing | PyTest + Postman | Module interactions: auth → order → slot → payment |
| API Testing | Postman Collections | All REST endpoints with positive and negative cases |
| Load / Performance Testing | Apache JMeter | Simulate 200+ concurrent lunch-hour users |
| UI Testing | Selenium / Appium | End-to-end customer and baker workflows |
| User Acceptance Testing | Real Users | Student and baker volunteers test on staging environment |

## 11.2 Critical Test Cases

| Test ID | Description | Expected Result |
|---|---|---|
| UT-01 | Valid login with correct credentials | JWT token returned; role confirmed |
| UT-02 | Login with incorrect password | 401 error with descriptive message |
| UT-05 | SQL injection attempt on login | Login blocked; no data exposed |
| IT-01 | Order creation with valid slot and stock | Order created; slot count incremented |
| IT-02 | Order creation when slot is FULL | 409 error; 'Slot Full' response |
| IT-03 | Concurrent orders for last available slot | Exactly one order succeeds; other rejected |
| LT-01 | 200 concurrent users during lunch slot | Response time < 2s; no data corruption |
| ST-01 | Customer attempts baker-only endpoint | 403 Forbidden response |

# 13. Future Scalability & Roadmap

## 13.1 Phase 1 — Core System (Current)

- User authentication with JWT and RBAC
- Time-slot-based ordering with concurrency-safe algorithm
- Order lifecycle management with real-time status updates
- Basic admin dashboard with user, baker, and slot management
- Payment gateway integration (online + cash)

## 13.2 Phase 2 — Intelligence Layer

- AI Demand Prediction: ML models (ARIMA, XGBoost) trained on historical order data to forecast peak slots, high-demand items, and required quantities per day
- Dynamic Capacity Adjustment: Real-time slot capacity calculated from staff count and item preparation time
- Food Waste Analytics: Automated alerts when predicted demand deviates significantly from current inventory

## 13.3 Phase 3 — Multi-Canteen Ecosystem

- Multi-canteen support with single login and unified order tracking
- Canteen performance benchmarking and comparison dashboard
- Mobile app push notifications for order ready alerts
- QR code-based order pickup verification
- Student meal subscription plans with auto-ordering

## 13.4 Scalability Architecture

| Concern | Current Approach | Future Scaling Strategy |
|---|---|---|
| Database | Single PostgreSQL instance | Read replicas + connection pooling (PgBouncer) |
| API Server | Single FastAPI instance | Horizontal scaling with load balancer (Nginx) |
| Caching | None (DB queries) | Redis cache for menu and slot availability |
| Notifications | Synchronous | Async message queue (Celery + Redis) |
| ML Predictions | Not yet built | Separate ML microservice (Python + scikit-learn) |

# 14. Implementation Plan

## 14.1 Development Phases & Timeline

| Phase | Duration | Deliverables |
|---|---|---|
| Phase 1: Foundation | Week 1–2 | Project setup, DB schema, FastAPI skeleton, JWT auth |
| Phase 2: Core Modules | Week 3–5 | Authentication, Order Management, Slot Algorithm |
| Phase 3: Frontend Development | Week 6–8 | Customer app, Baker dashboard, Admin portal |
| Phase 4: Integration & Testing | Week 9–10 | End-to-end integration, unit tests, load tests |
| Phase 5: UAT & Refinement | Week 11 | Real-user testing, bug fixes, performance tuning |
| Phase 6: Deployment & Handover | Week 12 | Server deployment, documentation, training |

## 14.2 Resource Requirements

### Hardware

- Server: Minimum 8GB RAM, 4-core CPU, 100GB SSD storage
- Client Devices: Android (5.0+), iOS (12.0+), or modern web browser
- Network: Campus WiFi coverage at canteen and pickup areas

### Software & Tools

- Python 3.11+ with FastAPI, SQLAlchemy, Pydantic, bcrypt
- PostgreSQL 15+ for production database
- React / React Native for frontend development
- PyTest, Postman, JMeter for testing
- Git for version control; GitHub Actions for CI/CD pipeline

# 15. Conclusion

AuraBake represents a well-researched, technically sound, and practically impactful solution to a problem that affects every student on campus every single day. The project is not merely an application — it is a demonstration of how fundamental computing principles (scheduling algorithms, database transactions, role-based security, and RESTful architecture) can be applied to create meaningful real-world impact.

By eliminating queues, reducing food waste, empowering kitchen staff with advance visibility, and giving administrators actionable data, AuraBake creates a win for every stakeholder in the campus canteen ecosystem.

The system is architected for growth: starting as a campus pilot and scalable to a multi-canteen, AI-powered food management platform. The technical foundation laid in Phase 1 is designed to support the intelligence and scalability features planned for Phases 2 and 3.

## 15.1 Request to College Management

We respectfully request the college management's approval and support for the AuraBake project, including:

- Access to campus canteen operational data for demand analysis
- Permission to conduct a pilot trial with student and baker volunteers
- Server/hosting infrastructure support for deployment
- Faculty mentor assignment for academic supervision
- Permission to survey students for UAT feedback collection

**AuraBake — Baked Fresh. Ordered Smart.**

*A College Pre-Order Food System*

Document Version 1.0 · April 2026 · Confidential